

第 06 讲: Kernel、Triton 与 XLA

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	4
1.1 本章知识路线	4
1.2 结构图	4
2 本章主线	4
2.1 本章要解决的核心问题	5
2.2 从机制到资源账本	5
2.3 从课堂讲解到可复现实验	6
3 术语、符号与问题边界	6
3.1 术语之间的关系	6
4 Benchmark/profiling 的基本闭环	6
4.1 为什么要讨论 Benchmark/profiling 的基本闭环	7
4.2 Benchmark/profiling 的基本闭环的机制	7
4.3 Benchmark/profiling 的基本闭环的数学表达	7
4.4 从 Benchmark/profiling 的基本闭环到程序	7
4.5 边界条件与常见误解	7
4.6 与本章其他概念的关系	8
4.7 怎样检验 Benchmark/profiling 的基本闭环	8
4.8 概念辨析: Benchmark/profiling 的基本闭环	8
4.9 Benchmark/profiling 的基本闭环的小结	8
5 Triton 的 programming model	9
5.1 为什么要讨论 Triton 的 programming model	9
5.2 Triton 的 programming model 的机制	9
5.3 Triton 的 programming model 的数学表达	9
5.4 从 Triton 的 programming model 到程序	9
5.5 边界条件与常见误解	10

5.6	与本章其他概念的关系	10
5.7	怎样检验 Triton 的 programming model	10
5.8	概念辨析: Triton 的 programming model	10
5.9	Triton 的 programming model 的小结	10
6	Fusion、tiling 与 compiler	11
6.1	为什么要讨论 Fusion、tiling 与 compiler	11
6.2	Fusion、tiling 与 compiler 的机制	11
6.3	Fusion、tiling 与 compiler 的数学表达	11
6.4	从 Fusion、tiling 与 compiler 到程序	11
6.5	边界条件与常见误解	12
6.6	与本章其他概念的关系	12
6.7	怎样检验 Fusion、tiling 与 compiler	12
6.8	概念辨析: Fusion、tiling 与 compiler	12
6.9	Fusion、tiling 与 compiler 的小结	13
7	综合讨论: 从局部机制到完整系统	13
7.1	Benchmark/profiling 的基本闭环在系统中的位置	13
7.2	Triton 的 programming model 在系统中的位置	13
7.3	Fusion、tiling 与 compiler 在系统中的位置	13
7.4	把本章知识用于读论文和复现实验	14
8	例题: 把概念变成可计算检查	14
8.1	Triton row-wise softmax 的稳定实现	14
9	实验设计: 从小例子到可复现结论	14
9.1	最小输入	15
9.2	资源账本	15
9.3	单变量消融	15
9.4	失败条件	15
9.5	记录与报告	15
9.6	从小实验到大结论	15
9.7	把本章重新读成一条证据链	15
10	课程资料与回看路径	16
10.1	视频回看路径	16
10.2	课程资料阅读路径	16
11	实践说明、历史位置与风险	17

12 复习要点	17
12.1 综合自测	18
13 总结与延伸	18
13.1 本章小结	18
13.2 延伸解释	18
13.3 练习	18

1 本章导读

本章讨论 **Kernel、Triton 与 XLA**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想一个 row-wise softmax kernel 比 PyTorch baseline 慢。真正的修复流程不是凭直觉改代码，而是先做 warmup 和同步计时，再用 profiler 找瓶颈，最后通过 tiling、fusion 或 Triton block program 控制数据移动。本章对应 CS336 Spring 2026 第 06 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

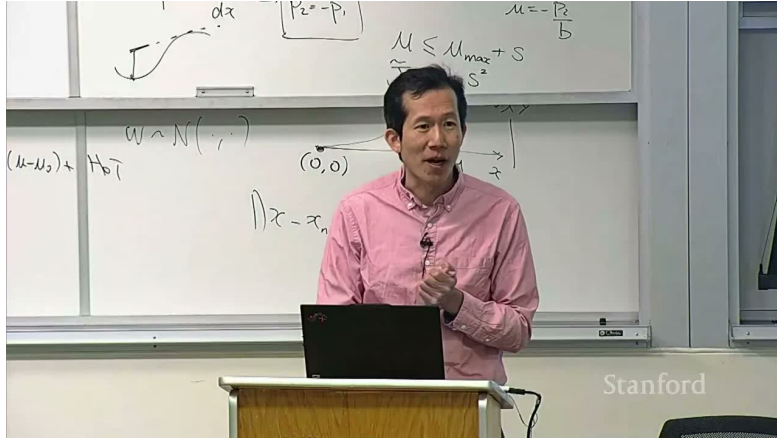


图 1: 第 06 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Kernel、Triton 与 XLA 为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Kernel、Triton 与 XLA 的主线是把 language model 的抽象计算放回真实机器：哪些张量要移动，哪些状态要保存，哪些瓶颈会在训练或服务时显现。

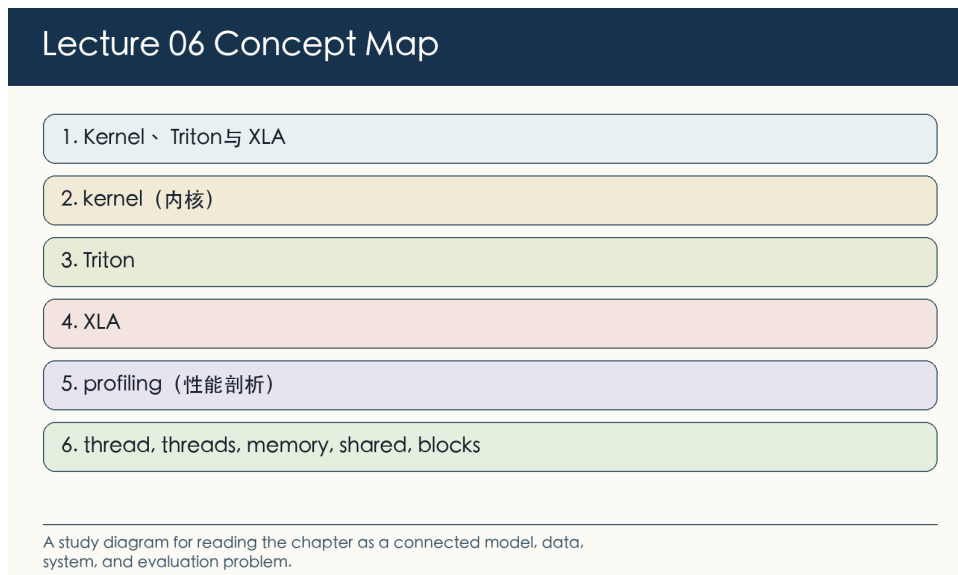


图 2: 第 06 讲概念图: 本章问题、术语和机制的关系。

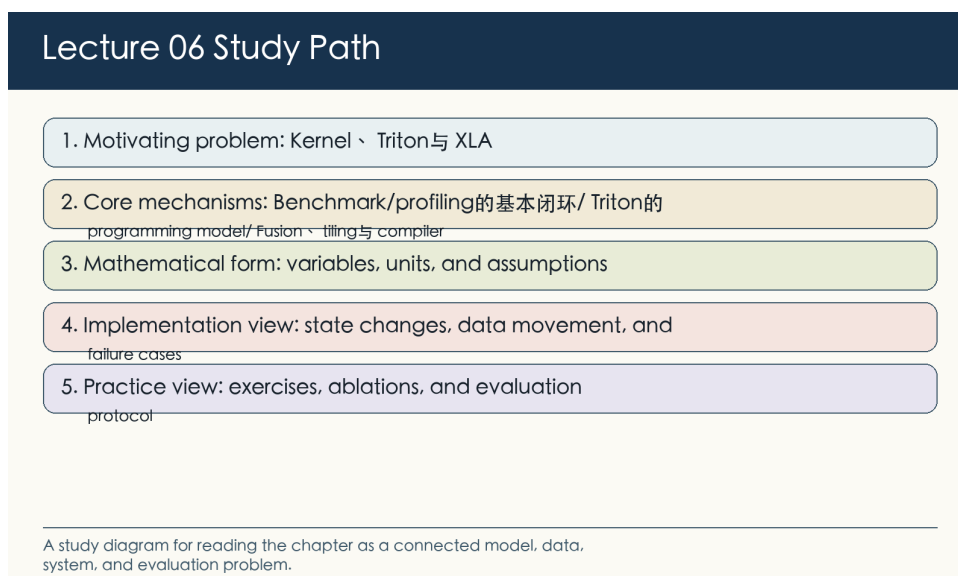


图 3: 第 06 讲学习路径: 从问题定义进入公式、实现、实验和练习。

2.1 本章要解决的核心问题

本讲从 profiling 和 benchmarking 出发, 要求先测量再优化。GPU 异步执行意味着计时必须 warmup、同步, 并用 profiler 找到真正热点。读完这一段后, 应能用一两句话说明 Kernel、Triton 与 XLA 要解决的瓶颈, 并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

Triton 的教学价值在于把 GPU kernel 写作提升到 block-level 抽象: 显式处理 offsets、mask、load、compute、store, 同时仍能思考 coalescing、shared memory 和 tiling。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

kernel fusion 和 tiling 是减少 HBM 往返的基本策略。它们提高性能的前提是没有引入过高寄存器压力、过低 occupancy 或数值错误。最后把课堂讲解转成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	kernel（内核）	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
2	Triton	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	XLA	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	profiling（性能剖析）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	fusion（算子融合）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	tiling（分块）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，kernel（内核）给出问题入口，Triton 描述主要机制，XLA 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 thread、memory、shared、threads、block、triton 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 Benchmark/profiling 的基本闭环

现在进入 Benchmark/profiling 的基本闭环。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

4.1 为什么要讨论 Benchmark/profiling 的基本闭环

课程材料明确把本讲定位为 GPU 高层概览后的代码层深入：写 Triton kernels、benchmark 和 profiling。benchmark 回答端到端多快，profiling 回答时间花在哪里；两者必须迭代使用。正确计时 GPU 代码需要 warmup、CUDA events 或同步，否则测到的可能只是 CPU launch time。Benchmark/profiling 的基本闭环把 Kernel、Triton 与 XLA 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

4.2 Benchmark/profiling 的基本闭环的机制

课程材料明确把本讲定位为 GPU 高层概览后的代码层深入：写 Triton kernels、benchmark 和 profiling。在 Benchmark/profiling 的基本闭环中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。benchmark 回答端到端多快，profiling 回答时间花在哪里；两者必须迭代使用。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。正确计时 GPU 代码需要 warmup、CUDA events 或同步，否则测到的可能只是 CPU launch time。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Benchmark/profiling 的基本闭环的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 Benchmark/profiling 的基本闭环的数学表达

$$\bar{t} = \frac{1}{n} \sum_{i=1}^n t_i, \quad \text{speedup} = \frac{t_{\text{baseline}}}{t_{\text{optimized}}}$$

符号说明： $\bar{t}$ 是多次 trial 的平均时间；speedup 要在同一输入、同一硬件、同一 dtype 下比较。这个关系式应被读成 Benchmark/profiling 的基本闭环的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 从 Benchmark/profiling 的基本闭环到程序

把 Benchmark/profiling 的基本闭环写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
for _ in range(warmups): run()
synchronize()
for trial in trials:
    start_event.record(); run(); end_event.record(); synchronize()
```

阅读这段实现时，重点看 Benchmark/profiling 的基本闭环怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- 一次 benchmark 不足以说明 scaling；要扫维度、batch 和 dtype。
- profiling 结果依赖输入 shape 和编译状态。

- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

Benchmark/profiling 的基本闭环连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

4.7 怎样检验 Benchmark/profiling 的基本闭环

检验 Benchmark/profiling 的基本闭环时，先把抽象说法落到硬件和系统状态上。与 benchmark、profiling、CUDA event、warmup、speedup 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\bar{t}$ 是多次 trial 的平均时间；speedup 要在同一输入、同一硬件、同一 dtype 下比较。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：一次 benchmark 不足以说明 scaling；要扫维度、batch 和 dtype；profiling 结果依赖输入 shape 和编译状态。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Kernel、Triton 与 XLA 时，要说明 Benchmark/profiling 的基本闭环改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

4.8 概念辨析：Benchmark/profiling 的基本闭环

benchmark、profiling、CUDA event、warmup、speedup 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Benchmark/profiling 的基本闭环复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

4.9 Benchmark/profiling 的基本闭环的小结

- 讨论 Benchmark/profiling 的基本闭环时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

5 Triton 的 programming model

现在进入 Triton 的 programming model。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论 Triton 的 programming model

Triton 让用户以 block 为单位写 kernel，显式处理 offsets、mask、load、compute、store。elementwise GeLU、row-wise softmax、row sum 和 matmul+ReLU 是课程材料中的核心例子。高性能 kernel 的关键是减少 HBM 往返、利用 shared/on-chip memory、控制 block size 并让访存合并。Triton 的 programming model 把 Kernel、Triton 与 XLA 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

5.2 Triton 的 programming model 的机制

Triton 让用户以 block 为单位写 kernel，显式处理 offsets、mask、load、compute、store。在 Triton 的 programming model 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。elementwise GeLU、row-wise softmax、row sum 和 matmul+ReLU 是课程材料中的核心例子。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。高性能 kernel 的关键是减少 HBM 往返、利用 shared/on-chip memory、控制 block size 并让访存合并。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Triton 的 programming model 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 Triton 的 programming model 的数学表达

$$\text{softmax}(x_i) = \frac{\exp(x_i - \max_j x_j)}{\sum_j \exp(x_j - \max_k x_k)}$$

符号说明：减去最大值是数值稳定技巧；softmax kernel 通常需要 reduction 和 row-wise normalization。这个关系式应被读成 Triton 的 programming model 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 从 Triton 的 programming model 到程序

把 Triton 的 programming model 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
pid = program_id(0)
offsets = pid * BLOCK + arange(0, BLOCK)
x = load(ptr + offsets, mask=offsets < n)
y = gelu(x)
store(out + offsets, y, mask=offsets < n)
```

阅读这段实现时，重点看 Triton 的 programming model 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- Triton 代码可读性高于手写 PTX/CUDA，但仍需要理解硬件。
- mask 处理错误会带来 silent correctness bugs。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

Triton 的 programming model 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

5.7 怎样检验 Triton 的 programming model

检验 Triton 的 programming model 时，先把抽象说法落到硬件和系统状态上。与 Triton、GeLU、softmax、mask、block 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：减去最大值是数值稳定技巧；softmax kernel 通常需要 reduction 和 row-wise normalization。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：Triton 代码可读性高于手写 PTX/CUDA，但仍需要理解硬件；mask 处理错误会带来 silent correctness bugs。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Kernel、Triton 与 XLA 时，要说明 Triton 的 programming model 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

5.8 概念辨析：Triton 的 programming model

Triton、GeLU、softmax、mask、block 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，Triton 让课程把 kernel 优化写成可读的 block-level 程序。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Triton 的 programming model 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

5.9 Triton 的 programming model 的小结

- 讨论 Triton 的 programming model 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。

- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

6 Fusion、tiling 与 compiler

现在进入 Fusion、tiling 与 compiler。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 Fusion、tiling 与 compiler

kernel fusion 把多个 elementwise/reduction 操作合并，减少中间张量写回 HBM。tiling 让 matmul 或 attention 在片上内存中复用数据，是 FlashAttention 等算法的核心系统思想。XLA/torch.compile 等 compiler 可以自动做一部分融合，但手写 kernel 仍常用于热点路径。Fusion、tiling 与 compiler 把 Kernel、Triton 与 XLA 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 Fusion、tiling 与 compiler 的机制

kernel fusion 把多个 elementwise/reduction 操作合并，减少中间张量写回 HBM。在 Fusion、tiling 与 compiler 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。tiling 让 matmul 或 attention 在片上内存中复用数据，是 FlashAttention 等算法的核心系统思想。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。XLA/torch.compile 等 compiler 可以自动做一部分融合，但手写 kernel 仍常用于热点路径。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Fusion、tiling 与 compiler 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 Fusion、tiling 与 compiler 的数学表达

$$\text{HBM traffic}_{\text{fused}} < \sum_i \text{HBM traffic}_i$$

符号说明：融合的收益来自减少中间结果读写；但如果融合降低 occupancy 或增加寄存器压力，也可能不划算。这个关系式应被读成 Fusion、tiling 与 compiler 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从 Fusion、tiling 与 compiler 到程序

把 Fusion、tiling 与 compiler 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
for q_tile in Q:
    for k_tile, v_tile in KV:
        update_online_softmax(q_tile, k_tile, v_tile)
```

`write_output_once()`

阅读这段实现时，重点看 Fusion、tiling 与 compiler 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- fusion 会增加 kernel 复杂度，debug 和数值验证成本上升。
- compiler 生成的 kernel 需要 profile 验证，不能默认最优。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

Fusion、tiling 与 compiler 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

6.7 怎样检验 Fusion、tiling 与 compiler

检验 Fusion、tiling 与 compiler 时，先把抽象说法落到硬件和系统状态上。与 fusion、tiling、FlashAttention、XLA、compiler 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：融合的收益来自减少中间结果读写；但如果融合降低 occupancy 或增加寄存器压力，也可能不划算。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：fusion 会增加 kernel 复杂度，debug 和数值验证成本上升；compiler 生成的 kernel 需要 profile 验证，不能默认最优。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Kernel、Triton 与 XLA 时，要说明 Fusion、tiling 与 compiler 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：Fusion、tiling 与 compiler

fusion、tiling、FlashAttention、XLA、compiler 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，attention（注意力）提供 token 间交互，但长上下文下成本很高。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Fusion、tiling 与 compiler 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 Fusion、tiling 与 compiler 的小结

- 讨论 Fusion、tiling 与 compiler 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是 Kernel、Triton 与 XLA，但它不应被读成几个相邻知识点的拼接。Benchmark/profiling 的基本闭环、Triton 的 programming model、Fusion、tiling 与 compiler 共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 kernel（内核）、Triton、XLA、profiling（性能剖析）、fusion（算子融合）、tiling（分块）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 Benchmark/profiling 的基本闭环在系统中的位置

Benchmark/profiling 的基本闭环首先提供一个局部解释：课程材料明确把本讲定位为 GPU 高层概览后的代码层深入：写 Triton kernels、benchmark 和 profiling。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Benchmark/profiling 的基本闭环会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Benchmark/profiling 的基本闭环不能脱离边界条件理解。一个关键 caveat 是：一次 benchmark 不足以说明 scaling；要扫维度、batch 和 dtype。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Triton 的 programming model 在系统中的位置

Triton 的 programming model 首先提供一个局部解释：Triton 让用户以 block 为单位写 kernel，显式处理 offsets、mask、load、compute、store。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Triton 的 programming model 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Triton 的 programming model 不能脱离边界条件理解。一个关键 caveat 是：Triton 代码可读性高于手写 PTX/CUDA，但仍需要理解硬件。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 Fusion、tiling 与 compiler 在系统中的位置

Fusion、tiling 与 compiler 首先提供一个局部解释：kernel fusion 把多个 elementwise/reduction 操作合并，减少中间张量写回 HBM。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Fusion、tiling 与 compiler 会改变至

少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Fusion、tiling 与 compiler 不能脱离边界条件理解。一个关键 caveat 是：fusion 会增加 kernel 复杂度，debug 和数值验证成本上升。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 Triton row-wise softmax 的稳定实现

一行 logits 需要做 softmax。直接 $\exp(x)$ 可能溢出，分多个 kernel 又会多次访问 HBM。

- 先在 block 内求 $m=\max(x)$ ，计算 $\exp(x-m)$ 。
- 再求和并归一化，把读、reduction、写尽量放在一个 kernel 中。
- mask 必须正确处理越界元素，否则会产生 silent numerical bugs。

结论。 Triton 的价值在于把数值稳定性和内存访问模式放在同一个可读程序里。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Kernel、Triton 与 XLA 不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Kernel、Triton 与 XLA 重新读成一条证据链：先用“Benchmark/profiling 的基本闭环”确定问题入口，再用“Triton 的 programming model”解释主要机制，最后用“Fusion、tiling 与 compiler”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、

变量和证据。本章反复使用的术语包括 kernel（内核）、Triton、XLA、profiling（性能剖析）、fusion（算子融合）、tiling（分块）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:07:20	thread, then, going, there's, memory, threads, think	回看定义、例子、推导或 caveat 在该段中的位置。
00:21:09-00:28:32	then, going, your, time, profiling, it's, think	回看定义、例子、推导或 caveat 在该段中的位置。
00:43:33-00:50:41	going, then, it's, blocksize, block, memory, yeah	回看定义、例子、推导或 caveat 在该段中的位置。
00:58:06-01:05:21	going, then, number, softmax, think, row., that's	回看定义、例子、推导或 caveat 在该段中的位置。
01:19:28-01:26:30	then, going, there's, write, think, time, tile	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Last lecture: high-level overview of GPUs and performance	cache, shared, memory, size, bandwidth, performance, triton
official_03	TTTT TTTT	threads, thread, bank, warp, registers, same, multiple
official_05	You can use ...	time, pytorch, recipes, cuda, benchmark, https, html

official_08	- Multiply and accumulate.	tile, shared, memory, arithmetic, intensity, output, accumulate
official_10	Function: triton_row_sum	function, triton_row_sum, row_sum_kernel, triton_matmul_relu_example, naive_matmul_relu, triton_matmul_relu, matmul_relu_kernel

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Kernel、Triton 与 XLA 在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。
- 系统风险 (systems risk): 性能剖析结果依赖硬件、driver、kernel、shape 和 batch distribution；脱离 workload 的性能数字不可直接复用。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	kernel (内核)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 2	Triton	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

条目 3	XLA	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 4	profiling（性能剖析）	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 5	fusion（算子融合）	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 6	tiling（分块）	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- Kernel、Triton 与 XLA 的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。

- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。